Laundry Booking

Aplicație de rezervări pentru spălătoria din cămin, cu arhitectură event-driven și agent AI
PCBE – Stage 2 · propunere de proiect

1. Descrierea aplicației

Problema. În cămin sunt puține mașini de spălat și uscătoare. Studenții ajung și găsesc mașina ocupată sau stricată, unii rezervă și nu mai vin, iar administratorul nu are o imagine clară despre ce mașini se defectează des.

Soluția. Studenții rezervă sloturi pe mașini și raportează defecte. Aplicația detectează rezervările neonorate (no-show) și trimite notificări. Un agent AI analizează zilnic defectele și no-show-urile și **propune** mentenanță sau penalizări, pe care administratorul le aprobă sau le respinge.

Funcționalități principale

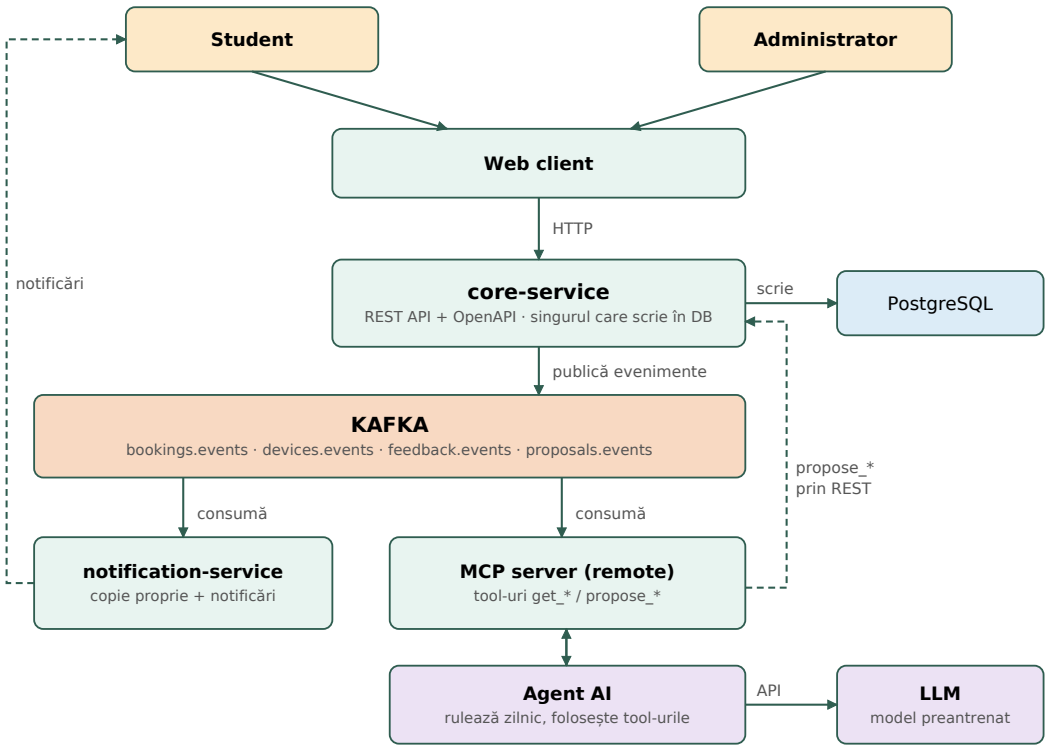
- rezervare și anulare de sloturi pe mașini de spălat și uscătoare
- raportare de defecte și sesizări de la studenți
- detectarea rezervărilor neonorate
- notificări pentru studenți
- agent AI care propune acțiuni, cu aprobare finală din partea adminului

Limite (ce nu face). Fără plăți și fără conectare la mașini reale. Autentificarea e simplă, iar datele sunt de test.

2. Actori

Student	Vede mașinile și sloturile libere, face și anulează rezervări, confirmă începutul ciclului, raportează defecte, trimite sesizări, primește notificări.
Administrator	Vede rezervările, mașinile și defectele, gestionează mașinile, aprobă sau respinge propunerile agentului, citește rezumatul sesizărilor.
Agent AI (automat)	Rulează zilnic, citește defectele și no-show-urile prin tool-uri MCP și creează propuneri. Nu aprobă și nu modifică nimic direct.
Sistem de notificări (automat)	Ascultă evenimentele din Kafka și trimite studenților notificări (slot care începe, rezervare anulată, defect rezolvat).

3. Arhitectura



- **core-service** este singurul care scrie în baza de date (single source of truth) și publică evenimente pe Kafka.
- **notification-service** și **MCP server** își construiesc propriile copii de citire din evenimente.
- Agentul citește prin tool-urile MCP, iar propunerile ajung prin API-ul REST la core-service, unde adminul le aprobă.

4. Tehnologii propuse

Message broker	Apache Kafka (păstrează evenimentele și permite reconstruirea copiilor de date)
API	REST, documentat cu OpenAPI
Bază de date	PostgreSQL, accesată doar de core-service
MCP	Server MCP remote, cu tool-uri de citire și de propunere
AI	LLM preantrenat folosit prin API, fără antrenare proprie
Deploy	Docker și docker-compose, câte un container pe serviciu

5. Evenimente Kafka și tool-uri MCP

bookings.events	`booking.created, booking.cancelled, booking.started, booking.noshow`
devices.events	`fault.reported, maintenance.opened, maintenance.closed`
feedback.events	`feedback.submitted`
proposals.events	`proposal.created, proposal.approved, proposal.rejected`

Tool-uri de citire	`get_device_errors(since, building)`
	`get_bookings(since)`
	`get_noshows(since)`

Tool-uri de propunere	`propose_maintenance(device_id, reason, severity)` `propose_penalty(user_id, reason)`

6. Fluxuri principale

Rezervare. Studentul rezervă din web, iar core-service salvează rezervarea și publică `booking.created`. Notification-service primește evenimentul, își actualizează copia și programează notificarea.

Agentul zilnic. Agentul apelează tool-urile de citire pentru defectele și no-show-urile din ziua precedentă, le grupează pe mașină și pe student și apelează tool-urile de propunere. Adminul vede propunerile în panou și le aprobă sau le respinge, iar decizia se publică în `proposals.events`.

7. Cerințele Stage 2

Cerință	Cum o acoperim
Minim 3 componente remote	core-service, notification-service, MCP server + agent, web client
API-uri OpenAPI	core-service expune un API REST documentat
Single source of truth	doar core-service scrie în DB, restul își fac copii din evenimente
Broker	Kafka
Docker	docker-compose cu toate serviciile
Server MCP remote	5 tool-uri: 3 de citire, 2 de propunere
Agent cu task concret	analiza zilnică a defectelor și no-show-urilor, cu propuneri pentru admin